



Programmation fonctionnelle (OCAML)

Mouhamadou GAYE

Enseignant-Chercheur
Département d'Informatique
Master en Informatique
Option Génie Logiciel

5 mai 2025



Objectifs du cours

- Comprendre les principes de la programmation fonctionnelle ;



Objectifs du cours

- Comprendre les principes de la programmation fonctionnelle ;
- Décrire la syntaxe et la sémantique du langage OCAML ;



Objectifs du cours

- Comprendre les principes de la programmation fonctionnelle ;
- Décrire la syntaxe et la sémantique du langage OCAML ;
- Utiliser des fonctions de haut niveau de manière optimale pour développer des programmes, essentiellement récursifs ;



Objectifs du cours

- Comprendre les principes de la programmation fonctionnelle ;
- Décrire la syntaxe et la sémantique du langage OCAML ;
- Utiliser des fonctions de haut niveau de manière optimale pour développer des programmes, essentiellement récursifs ;
- Écrire des applications maintenables, réutilisables, lisibles, modulaires.



- Chapitre 1 : Éléments de base du OCAML
- Chapitre 2 : Structures de données
- Chapitre 3 : Modules et Classes



Évaluation

- Projet : **50%**
- Examen final : **50%**



Chapitre 1 : Éléments de base du OCAML



- **Programmation impérative**



- **Programmation impérative**

- Définition d'une liste d'instructions qui déterminent les modifications successives des états de la mémoire pour produire le résultat demandé.



- **Programmation impérative**

- Définition d'une liste d'instructions qui déterminent les modifications successives des états de la mémoire pour produire le résultat demandé.
- Utilisation de structures de contrôle et des structures de données.



- **Programmation impérative**

- Définition d'une liste d'instructions qui déterminent les modifications successives des états de la mémoire pour produire le résultat demandé.
- Utilisation de structures de contrôle et des structures de données.
- Usage intensif de l'itération.



- **Programmation impérative**

- Définition d'une liste d'instructions qui déterminent les modifications successives des états de la mémoire pour produire le résultat demandé.
- Utilisation de structures de contrôle et des structures de données.
- Usage intensif de l'itération.

- **Programmation déclarative**

- Un style de programmation où l'on décrit les relations qui existent entre les données et les résultats.



- **Programmation impérative**

- Définition d'une liste d'instructions qui déterminent les modifications successives des états de la mémoire pour produire le résultat demandé.
- Utilisation de structures de contrôle et des structures de données.
- Usage intensif de l'itération.

- **Programmation déclarative**

- Un style de programmation où l'on décrit les relations qui existent entre les données et les résultats.



- **Programmation impérative**

- Définition d'une liste d'instructions qui déterminent les modifications successives des états de la mémoire pour produire le résultat demandé.
- Utilisation de structures de contrôle et des structures de données.
- Usage intensif de l'itération.

- **Programmation déclarative**

- Un style de programmation où l'on décrit les relations qui existent entre les données et les résultats.
- **Programmation logique** : les relations sont des formules logiques et le traitement est effectué par inférence.



- **Programmation impérative**

- Définition d'une liste d'instructions qui déterminent les modifications successives des états de la mémoire pour produire le résultat demandé.
- Utilisation de structures de contrôle et des structures de données.
- Usage intensif de l'itération.

- **Programmation déclarative**

- Un style de programmation où l'on décrit les relations qui existent entre les données et les résultats.
- **Programmation logique** : les relations sont des formules logiques et le traitement est effectué par inférence.
- **Programmation fonctionnelle** : les relations sont assimilées à des fonctions et le traitement est une combinaison de fonctions intermédiaires.



Concepts de la programmation fonctionnelle

- **Fonctions pures** : des fonctions qui ne dépendent que de leurs propres paramètres.



Concepts de la programmation fonctionnelle

- **Fonctions pures** : des fonctions qui ne dépendent que de leurs propres paramètres.
- **Immutabilité** : les fonctions ne changent pas d'état, sont utilisées comme des constantes.



Concepts de la programmation fonctionnelle

- **Fonctions pures** : des fonctions qui ne dépendent que de leurs propres paramètres.
- **Immutabilité** : les fonctions ne changent pas d'état, sont utilisées comme des constantes.
- **Ecriture lazy** : exécuter les expressions uniquement lorsqu'elles sont indispensables.



Concepts de la programmation fonctionnelle

- **Fonctions pures** : des fonctions qui ne dépendent que de leurs propres paramètres.
- **Immutabilité** : les fonctions ne changent pas d'état, sont utilisées comme des constantes.
- **Ecriture lazy** : exécuter les expressions uniquement lorsqu'elles sont indispensables.
- **Séparation et traitement de données** : fonctions pures pour traiter les données et une fonction impure pour les insertions et les extractions.



Concepts de la programmation fonctionnelle

- **Fonctions pures** : des fonctions qui ne dépendent que de leurs propres paramètres.
- **Immutabilité** : les fonctions ne changent pas d'état, sont utilisées comme des constantes.
- **Ecriture lazy** : exécuter les expressions uniquement lorsqu'elles sont indispensables.
- **Séparation et traitement de données** : fonctions pures pour traiter les données et une fonction impure pour les insertions et les extractions.
- **Fonctions anonymes** : offrent une meilleure lisibilité du code.



Concepts de la programmation fonctionnelle

- **Fonctions pures** : des fonctions qui ne dépendent que de leurs propres paramètres.
- **Immutabilité** : les fonctions ne changent pas d'état, sont utilisées comme des constantes.
- **Ecriture lazy** : exécuter les expressions uniquement lorsqu'elles sont indispensables.
- **Séparation et traitement de données** : fonctions pures pour traiter les données et une fonction impure pour les insertions et les extractions.
- **Fonctions anonymes** : offrent une meilleure lisibilité du code.
- **Composition** : construction et utilisation de fonctions pour élaborer tout le programme.



- CAML (Categorical Abstract Machine Language) est un langage de programmation fonctionnelle.



- CAML (Categorical Abstract Machine Language) est un langage de programmation fonctionnelle.
 - Un langage de choix dans les laboratoires de recherche pour :



- CAML (Categorical Abstract Machine Language) est un langage de programmation fonctionnelle.
 - Un langage de choix dans les laboratoires de recherche pour :
 - Un langage facile avec lequel on résout des problèmes difficiles.



- CAML (Categorical Abstract Machine Language) est un langage de programmation fonctionnelle.
 - Un langage de choix dans les laboratoires de recherche pour :
 - Un langage facile avec lequel on résout des problèmes difficiles.
 - Il est fortement typé : le typage est automatique et ne nécessite pas d'annotation.



- CAML (Categorical Abstract Machine Language) est un langage de programmation fonctionnelle.
 - Un langage de choix dans les laboratoires de recherche pour :
 - Un langage facile avec lequel on résout des problèmes difficiles.
 - Il est fortement typé : le typage est automatique et ne nécessite pas d'annotation.
 - Les instructions se terminent par ; ; et le système interactif affiche le résultat après chaque instruction.



- CAML (Categorical Abstract Machine Language) est un langage de programmation fonctionnelle.
 - Un langage de choix dans les laboratoires de recherche pour :
 - Un langage facile avec lequel on résout des problèmes difficiles.
 - Il est fortement typé : le typage est automatique et ne nécessite pas d'annotation.
 - Les instructions se terminent par ; ; et le système interactif affiche le résultat après chaque instruction.
- OCAML (Objective Caml) est une variante du langage Caml auquel il ajoute une couche orientée objet et un système de modules.



- Compilateur en ligne : <https://try.ocamlpro.com/>
- Visual Studio Code sur Windows



Types de base et opérateurs

- **int** : +, -, *, /, mod
- **float** : +., -., *., /.
- **bool** : &&, ||, not
- **char** :
- **string** : ^ (concaténation)
- **unit** : type dont l'unique valeur notée () est vide c'est le "void" du C.
- Opérateurs de comparaison (pour tous les types) : =, >, <, >=, <=, <>



Une définition permet d'associer une valeur à un nom pour pouvoir la réutiliser.



Une définition permet d'associer une valeur à un nom pour pouvoir la réutiliser.

- **Définition globale :**

let nom = expression | valeur ; ;

Exemple :

let n = 2 + 3 ; ;

val n : int = 5

NB : Une fois défini, un nom a toujours la même valeur, il n'est pas modifiable.



- **Définition locale :**

Une définition locale permet de définir des valeurs qui ne sont valides que pour le calcul en cours.

`let nom = expression | valeur in expression ; ;`

Exemple :

`let n = 3 in 7 - n ; ;`

`- : int = 4`

NB : Une définition locale est toujours suivie d'une expression et une expression peut contenir une définition locale.



- **Définition multiple :**

Une définition multiple est une succession de définitions (globales ou locales) séparées par le mot-clé **and**.

`let nom1 = expression1 | valeur1 and nom2 = expression2 | valeur2 ; ;`

Exemple :

```
let n = 3 and m = 5 ; ;  
val n : int = 3  
val m : int = 5  
let x = 2 and y = 4 in x + y ; ;  
- : int = 6
```

NB : Les définitions multiples se font en même temps et ne sont utilisables qu'une fois toutes les définitions terminées.



- **ref** permet de rendre une définition modifiable.

Exemple :

let x = ref 0 ; ;

- La référence ainsi définie se comporte comme une variable et peut être modifiée par l'opérateur `:=` et accédée par `!`

Exemple :

x := !x + 1 ; ;



- **Alternative**

Une alternative est une expression qui permet de faire des calculs dépendant d'une condition.

if condition then expression₁ else expression₂ ; ;

Exemple :

5 * (if true then 3 else 2) ; ;

- : int = 15

NB : expression₁ et expression₂ doivent être du même type.



Filtrage

Le filtrage consiste à mettre en correspondance des motifs et des expressions.



Filtrage

Le filtrage consiste à mettre en correspondance des motifs et des expressions.

- **Filtrage explicite**

match identifiant with

motif₁ → expression₁
| motif₂ → expression₂
|
| motif_n → expression_n ; ;

Exemple :

match x with

0 → "Zéro"
| 1 → "Un"
| 2 → "Deux" ; ;

NB : Les expressions doivent être du même type qui sera le type de l'identifiant.



- **Motif universel**

Le motif universel noté "`_`" est un motif qui s'approprie à toute valeur. Il peut être considéré comme le cas par défaut si toutes les valeurs ont été décrites.

match identifiant with

 motif₁ → expression₁

| motif₂ → expression₂

|

| `_` → expression_n ; ;



- **Motif conditionnel**

Le motif conditionnel permet de faire un filtre gardé c'est-à-dire un filtre qui dépend d'un test.

motif when condition $\rightarrow$ expression

Exemple :

match x with

0 $\rightarrow$ "Nul"

| y when y > 0 $\rightarrow$ "Strictement positif" ; ;



- **While**

Exemple :

```
let i = ref 10 in
while !i > 0 do
  print_int !i;
  i := !i - 1
done;;
```

- **for**

Exemple :

```
for i = 1 to 10 do
  print_int i;
done;;
```



- La séquence `expr1 ; expr2` signifie l'évaluation successive des deux expressions `expr1` puis `expr2`. La valeur de la construction est celle de `expr2`.



- La séquence `expr1 ; expr2` signifie l'évaluation successive des deux expressions `expr1` puis `expr2`. La valeur de la construction est celle de `expr2`.
- Pour exécuter une séquence dans une alternative, on encadre la séquence par des parenthèses ou par **begin** et **end**.

Exemple :

```
let parite n = if n mod 2 = 0 then
  begin
    print__int n ;
    print__string " est un nombre pair"
  end ; ;
```

NB : le `else` peut être omis si le type retour est `unit`.



- Fonctions à un argument

let identifiant argument = expression ; ;

Exemple :

let carre x = x * x ; ;

val carre : int → int = <fun>

Appel de la fonction :

carre 2 ; ;

- : int = 4

carre -1 ; ;

Error

carre (-1) ; ;

- : int = 1



- Fonctions à plusieurs arguments

let identifiant argument₁ argument₂ ... argument_n = expression ; ;

Exemple :

let perimetre x y = 2 * (x + y) ; ;

val perimetre : int → int → int = <fun>

Appel de la fonction :

perimetre 5 4 ; ;

- : int = 18

NB : $f \ x \ y$ n'équivaut pas à $f \ (x, y)$ et $f \ (g \ y)$ n'équivaut pas à $f \ g \ y$

Une fonction f à deux arguments est aussi une fonction à un argument, dont le résultat est une fonction



- Définition locale de fonction dans une expression

Exemple :

```
let cube x = x * x * x in cube 3 - cube 2;;
```

```
- : int = 19
```

```
cube 3;;
```

```
- : Error
```



- Définition locale de fonction dans une définition globale de fonction

Exemple :

```
let cube x = let carre y = y * y in x * carre x;;  
val cube : int → int = <fun>  
cube 3;;  
- : int = 27
```



- **Fonction anonyme**

let identifiant = function argument₁ → function argument₂ → ...
function argument_n → expression ; ;

Exemple :

```
let suivant = function x → x + 1 ; ;  
val suivant : int → int = <fun>  
suivant 2 ; ;  
- : int = 3
```



- **Fonction récursive**

let rec identifiant argument₁ argument₂ ... argument_n = expression ; ;

Exemple :

```
let rec fact n =  
    if n = 0 then  
        1  
    else  
        n * fact (n - 1) ;;  
val fact : int → int = <fun>  
fact 5 ;;  
- : int = 120
```



- **Fonction d'ordre supérieur**

Une fonction d'ordre supérieur ou fonctionnelle est une fonction dont les arguments ou le résultat sont eux-mêmes des fonctions.

Exemple :

Soit f une fonction dérivable. On approxime la fonction f' dérivée de f par la fonction qui à tout x associe $(f(x + h) - f(x)) / h$, avec h suffisamment petit.

let derivee f = let h = 0.001 in

function x $\rightarrow$ (f(x +. h) -. f(x)) /. h ; ;

derivee : (float $\rightarrow$ float) $\rightarrow$ float $\rightarrow$ float = <fun>



- **Fonction polymorphe**

Une fonction polymorphe est une fonction prenant un argument d'un type quelconque et renvoyant une valeur du même type.

Exemple :

```
let f x = x;;  
val f : 'a → 'a = <fun>
```



- Fonctions d'affichage et de lecture

- **print_int** pour afficher un entier ;
- **print_float** pour un réel ;
- **print_char** pour un caractère ;
- **print_string** pour une chaîne de caractères ;
- **print_newline ()** pour afficher une ligne vide ;
- **print_endline ()** pour afficher une chaîne de caractères et aller à la ligne ;
- **read_int ()** pour lire un entier

Les fonctions d'affichage retournent () de type unit.



- Les exceptions sont déclarées par le mot clef **exception** et le nom d'une exception doit commencer par une majuscule.



Exceptions

- Les exceptions sont déclarées par le mot clef **exception** et le nom d'une exception doit commencer par une majuscule.
- Elles sont levées par la fonction **raise** et on les traite avec la construction syntaxique **try...with**.



- Les exceptions sont déclarées par le mot clef **exception** et le nom d'une exception doit commencer par une majuscule.
- Elles sont levées par la fonction **raise** et on les traite avec la construction syntaxique **try...with**.
- Dans le bloc with on peut filtrer les différentes exceptions qui ont pu être levées.

NB : La fonction **failwith** peut être utilisée pour lever directement une exception.



Exceptions

Exemple :

```
exception Divparzéro ; ;
```

```
let div a b = if b = 0 then raise Divparzéro else a / b ; ;
```

ou

```
let div a b = if b = 0 then failwith "Division par zéro !" else a/b ; ;
```

